

Teknik Watermarking

Teknik watermark yang dipakai adalah **DCT** (Discrete Cosine Transform) dengan cara:

1. Membagi gambar menjadi blok berukuran 8*8 pixel.
2. Setiap blok akan diubah ke domain frekuensi menggunakan rumus DCT.
3. Watermark disisipkan ke koefisien frekuensi menengah. Koefisien frekuensi menengah dipilih karena cukup *invisible* namun masih tahan kompresi.
4. Untuk tugas kali ini, saya menggunakan watermark berupa **citra acak**
5. Setelah disisipkan watermark, gambar didekompresi kembali.

Langkah-Langkah Kode

Import

```
1 import argparse
2 import io
3 import os
4
5 import cv2
6 import matplotlib.pyplot as plt
7 import numpy as np
8 from PIL import Image
```

Pada bagian ini, dijalankan import untuk menambahkan library seperti:

- Argparse: membaca argument terminal
- io: buffer memory
- os: operasi file / folder
- cv2: untuk memproses gambar dan DCT
- matplotlib: untuk memplotting grafik outputnya nanti
- numpy: untuk men-generate matrix
- PIL: untuk menyimpan JPEG dengan Quality Factor (QF) tertentu sebagai output

Watermark Generation

```
15 def generate_binary_watermark(size=(32, 32)):
16     """Binary checkerboard watermark."""
17     wm = np.zeros(size, dtype=np.float32)
18     for i in range(size[0]):
19         for j in range(size[1]):
20             if (i + j) % 2 == 0:
21                 wm[i, j] = 1.0
22     return wm
```

Pada binary watermark generation, fungsi `generate_binary_watermark` akan membuat watermark checkerboard hitam-putih dengan cara membuat matrix kosong terlebih dahulu kemudian membuat pola kotak-kotak yang dapat dilihat pada loop.

```
def generate_random_watermark(size=(32, 32), seed=42):
    """Pseudo-random binary watermark (seed = secret key)."""
    rng = np.random.default_rng(seed)
    return rng.integers(0, 2, size=size).astype(np.float32)
```

Sedangkan pada random watermark generation, watermark dibuat secara random yang dapat dilihat pada bagian `rng = np.random.default_rng(seed)`.

DCT Embedding

```
MID_FREQ_POSITIONS = [(3, 2), (2, 3), (4, 1), (1, 4), (3, 3), (4, 2), (2, 4)]
```

Kemudian, bagian ini adalah untuk menentukan posisi frekuensi gambar yang akan diberi watermark yaitu pada area frekuensi menengah karena koefisien frekuensi menengah dipilih karena cukup *invisible* namun masih tahan kompresi.

```
def embed_dct(image_bgr, watermark, alpha=20):
    img = image_bgr.copy()
    ycrb = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb).astype(np.float32)
    Y = ycrb[:, :, 0]

    H, W = Y.shape
    wm_bits = watermark.flatten()
    num_bits = len(wm_bits)
```

Gambar kemudian diubah menjadi YCrCb karena watermark akan disisipkan pada channel Y (luminance) karena *brightness* (Y) lebih sensitif dan JPEG bekerja dominan di Y sehingga watermark akan lebih signifikan.

```
bit_idx = 0
blocks_used = 0

for row in range(0, H - 7, 8):
    for col in range(0, W - 7, 8):
        if bit_idx >= num_bits:
            break
        block = Y[row:row+8, col:col+8]
        dct_block = cv2.dct(block)

        pos = MID_FREQ_POSITIONS[bit_idx % len(MID_FREQ_POSITIONS)]
        bit = wm_bits[bit_idx]

        if bit == 1:
            dct_block[pos] += alpha
        else:
            dct_block[pos] -= alpha

        Y[row:row+8, col:col+8] = cv2.idct(dct_block)
        bit_idx += 1
        blocks_used += 1

ycrb[:, :, 0] = np.clip(Y, 0, 255)
watermarked_bgr = cv2.cvtColor(ycrb.astype(np.uint8), cv2.COLOR_YCrCb2BGR)
return watermarked_bgr, wm_bits, blocks_used
```

Setelah dipisah menjadi YCbCr, akan diloop per blok 8*8 untuk discan kemudian akan ditransformasi ke domain frekuensi (`dct_block = cv2.dct(block)`). Lalu, koefisien DCT akan dimodifikasi dengan menambahkan alpha. Terakhir, pada baris `cv2.idct(dct_block)`, domain frekuensi dikembalikan ke domain gambar.

JPEG Compression

```
def jpeg_compress(image_bgr, quality):
    img_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)
    pil_img = Image.fromarray(img_rgb)
    buffer = io.BytesIO()
    pil_img.save(buffer, format="JPEG", quality=quality)
    buffer.seek(0)
    decompressed = np.array(Image.open(buffer))
    return cv2.cvtColor(decompressed, cv2.COLOR_RGB2BGR)
```

Fungsi `jpeg_compress` berperan untuk mengompres JPEG dengan Quality Factor (QF) tertentu yang tertulis pada parameter `quality`. Jika QF kecil, maka kompresi menguat dan kualitas akan menurun, sedangkan jika QF besar, maka kompresi akan ringan dan kualitas output akan lebih bagus.

Evaluasi Kinerja Watermark

Saya mengubah Quality Factor (QF) menjadi beberapa nilai dari rentang QF=10 hingga QF=100 dengan interval 5 dan menghasilkan output berikut.

QF	BER	NC	Extractable?
10	0.4979	0.0041	NO
15	0.4955	0.0091	NO
20	0.4906	0.0188	NO
25	0.4725	0.0550	NO
30	0.3807	0.2385	NO
35	0.3230	0.3541	NO
40	0.2832	0.4335	NO
45	0.2523	0.4954	NO
50	0.1727	0.6547	NO
55	0.1380	0.7241	YES
60	0.0691	0.8618	YES
65	0.0686	0.8628	YES
70	0.0000	1.0000	YES
75	0.0000	1.0000	YES
80	0.0000	1.0000	YES
85	0.0000	1.0000	YES
90	0.0000	1.0000	YES
95	0.0000	1.0000	YES
100	0.0000	1.0000	YES

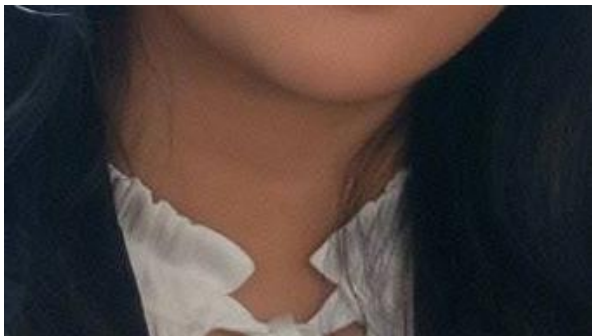
[RESULT] Watermark CAN be extracted at QF: [55, 60, 65, 70, 75, 80, 85, 90, 95, 100]
 [RESULT] Watermark CANNOT be extracted at QF: [10, 15, 20, 25, 30, 35, 40, 45, 50]
 [RESULT] => At QF <= 50, JPEG compression destroys the watermark.
 [INFO] Robustness plot saved.
 [INFO] Visual summary saved.

Dapat dilihat bahwa watermark dapat diekstrak mulai dari QF 55 hingga 100. Sedangkan untuk $QF \leq 50$ tidak dapat diekstrak. Outputnya, untuk $QF \geq 55$, watermark akan terlihat halus sedangkan jika $QF \leq 50$, gambar akan terlihat buram. Contohnya dapat dilihat pada gambar di bawah ini (gambar saya zoom supaya perbedaan terlihat lebih jelas):

QF 30 (tidak extractable)

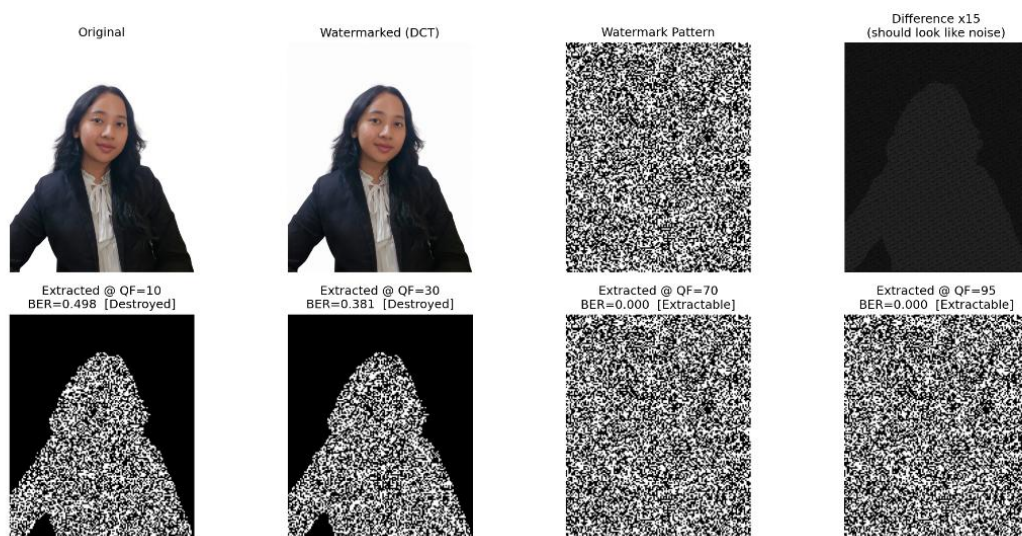


QF 70 (extractable)



Perbandingan output foto dengan berbagai nilai QF terdapat pada folder watermark_output

Visual Summary



Pada perbandingan hasil ekstraksi watermark pada berbagai level QF, dapat terlihat bahwa pada $QF \leq 50$ (misalnya pada $QF=10$ dan $QF=30$ yang memiliki label *destroyed*), hasil ekstraksi watermark membentuk siluet dari foto yang diimport. Hal ini disebabkan karena foto memiliki latar belakang putih polos sehingga pada area latar belakang memiliki frekuensi rendah, dan ketika di-*quantize* akan menjadi 0 dan watermark yang ditanam hilang sepenuhnya.

Di sisi lain, pada $QF \geq 55$ (misalnya pada $QF = 70$ dan $QF = 95$ yang memiliki label *extractable*), watermark dapat diekstraksi sepenuhnya karena di QF yang tinggi, nilai table kuantisasinya kecil sehingga hamper tidak ada koefisien yang dibuang.

Grafik Quality Factor

